Syed Naziya Parveen
2303A52394
BATCH - 42

Task 1 – Problem Identification & AI-Assisted Ideation

PROMPT:

As a product manager for a smart city, three problems are identified: manual complaint handling, parking issues, and poor waste management. The Civic Complaint Management System is selected. Currently, citizens face delays and no proper tracking when reporting issues. The problem is refined as the need for a digital system to register and track complaints easily. Key features include complaint submission, location tagging, status tracking, notifications, and admin dashboard. Target users are citizens, municipal staff, and workers. The expected impact is faster resolution, better transparency, and improved city management.

Code:

Lab23_tests.py U ×

Lab23_tests.py > ...

```python
"""Unit tests for Lab 23 capstone complaint management system."""

import os
import tempfile
import unittest

from Lab23_capstone.service import ComplaintService
from Lab23_capstone.storage import JSONStorage


class TestComplaintService(unittest.TestCase):
    """Validate complaint service behavior and edge cases."""

    def setUp(self) -> None:
        self.temp_dir = tempfile.TemporaryDirectory()
        self.data_file = os.path.join(self.temp_dir.name, "lab23_test_data.json")
        self.service = ComplaintService(JSONStorage(self.data_file))

    def tearDown(self) -> None:
        self.temp_dir.cleanup()

    def test_create_complaint_success(self) -> None:
        complaint = self.service.create_complaint(
            citizen_name="Ananya Rao",
            area="Ward 5",
            issue_type="Garbage Overflow",
            description="Bin has not been cleared for 4 days.",
            priority="High",
```

```python
class TestComplaintService(unittest.TestCase):
    def test_persistence_across_service_reloads(self) -> None:

        self.assertEqual(len(complaints), 1)
        self.assertEqual(complaints[0]["citizen_name"], "Rekha")

    def test_dashboard_metrics_counts(self) -> None:
        self.service.create_complaint(
            citizen_name="Ravi",
            area="Ward 1",
            issue_type="Garbage",
            description="Garbage pickup delayed.",
            priority="Critical",
        )
        self.service.create_complaint(
            citizen_name="Pooja",
            area="Ward 1",
            issue_type="Drainage",
            description="Drain blockage on main road.",
            priority="High",
        )

        self.service.update_status(1, "In Progress")
        self.service.update_status(2, "Resolved")

        metrics = self.service.dashboard_metrics()
```

Output :

```
Smart City Complaint and Waste Tracking Platform
1. Register new complaint
2. View all complaints
3. Filter complaints
4. Update complaint status
5. Assign staff
6. Dashboard metrics
7. Exit
Select an option: 1
Citizen Name: virat
Area/Ward: warangal
Issue Type (e.g., Garbage, Pothole): garbage
Description: not good
Select priority:
1. Low
2. Medium
3. High
4. Critical
Select option: 3
Complaint created successfully.
[#1] garbage | Area: warangal | Priority: High | Status: Open
Citizen: virat | Assigned: Unassigned | Created: 2026-04-12 13:37:10 | Updated: 2026-04-12 13:37:10
Description: not good
--------------------------------------------------------------------------------
```

Justification:

I chose the Civic Issue Reporting System because it solves a common problem faced by many citizens in communicating with municipal authorities. There is often a gap in reporting and tracking issues, which leads to delays and confusion. By digitizing this process, complaints can be tracked easily and handled more efficiently. It is a relatable and practical problem, making it suitable for a software project. It also provides clear benefits like better transparency and faster resolution.

Task 2 – System Planning & Architecture Design

PROMPT:

Act as a system architect. Design the architecture for a smart city complaint system called CivicFix. Suggest frontend, backend, and database components. Define the data flow clearly. Propose a basic database schema for users and complaints. Also include main APIs/modules. Keep the design simple and suitable for a prototype

Architecture & Schema (Text-based):

● **Frontend:** CLI (Command Line Interface) for prototype (can be extended to React.js web app later)
● **Backend:** Python application using modular functions (input handling, complaint management, status update)
● **Database:** In-memory Python lists/dictionaries (can be upgraded to SQLite for real use)●
Data Flow:
User enters complaint → Backend validates input → System generates Ticket ID → Data stored in dictionary → Confirmation shown to user

● **Database Schema:**
○ **Users:** {user_id: INT, name: STR, role: STR [citizen, admin]}
○ **Complaints:** {ticket_id: INT, citizen_name: STR, category: STR, description: STR, status: STR [Open, In-Progress, Resolved]}

● **APIs / Modules:**
○ register_complaint()
○ view_complaints()
○ update_status()
○ get_complaint_by_id()

Justification:

I chose this simple and modular architecture to enable quick development and testing. The CLI-based frontend makes it easy to implement within limited time, while the Python backend keeps the logic clear and organized. Using in-memory storage reduces complexity for the prototype, but it can be easily replaced with a real database later. This design is flexible, easy to understand,

and supports future expansion like adding a web interface or database without changing core logic.

Task 3 – Modular Code Development with AI

PROMPT:

Write a modular Python program for a CivicFix complaint system. Include input handling to register complaints, core logic to view and update complaint status, and data storage using a list of dictionaries. Add proper error handling and use functions for clean structure.

Code:

```python
# CivicFix Complaint Management System

complaints_db = []
ticket_counter = 1


def register_complaint(name, category, description):
    global ticket_counter

    try:
        name = name.strip()
        category = category.strip()
        description = description.strip()

        if not name or not category or not description:
            return "Error: All fields are required."

        complaint = {
            "ticket_id": ticket_counter,
            "citizen_name": name,
            "category": category,
            "description": description,
            "status": "Open"
        }
```

```python
        complaints_db.append(complaint)
        ticket_counter += 1

        return f"Success! Complaint registered with Ticket ID: {complaint['ticket_id']}"

    except Exception as e:
        return f"Error while registering complaint: {e}"


def view_complaints():
    try:
        if not complaints_db:
            return "No complaints found."

        output = "\n--- All Complaints ---\n"
        for complaint in complaints_db:
            output += (
                f"ID: {complaint['ticket_id']} | "
                f"Name: {complaint['citizen_name']} | "
                f"Category: {complaint['category']} | "
                f"Status: {complaint['status']}\n"
            )
        return output
```

```python
    except Exception as e:
        return f"Error while viewing complaints: {e}"


def update_status(ticket_id, new_status):
    valid_statuses = ["Open", "In-Progress", "Resolved"]

    try:
        ticket_id = int(ticket_id)

        if new_status not in valid_statuses:
            return "Error: Invalid status."

        for complaint in complaints_db:
            if complaint["ticket_id"] == ticket_id:
                complaint["status"] = new_status
                return f"Ticket {ticket_id} updated to {new_status}."

        return "Error: Ticket ID not found."

    except ValueError:
        return "Error: Ticket ID must be an integer."
    except Exception as e:
        return f"Error while updating status: {e}"
```

```python
        return f"Error while updating status: {e}"


def main():
    while True:
        print("\n--- CivicFix Complaint System ---")
        print("1. Register Complaint")
        print("2. View Complaints")
        print("3. Update Complaint Status")
        print("4. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            name = input("Enter citizen name: ")
            category = input("Enter complaint category: ")
            description = input("Enter complaint description: ")
            print(register_complaint(name, category, description))

        elif choice == "2":
            print(view_complaints())

        elif choice == "3":
```

```python
        elif choice == "3":
            ticket_id = input("Enter Ticket ID: ")
            new_status = input("Enter new status (Open/In-Progress/Resolved): ")
            print(update_status(ticket_id, new_status))

        elif choice == "4":
            print("Exiting CivicFix System.")
            break

        else:
            print("Invalid choice. Please try again.")


if __name__ == "__main__":
    main()
```

OUTPUT:

```
--- CivicFix Complaint System ---
1. Register Complaint
2. View Complaints
3. Update Complaint Status
4. Exit
Enter your choice: 1
Enter citizen name: naziya
Enter complaint category: road
Enter complaint description: open
Success! Complaint registered with Ticket ID: 1

--- CivicFix Complaint System ---
1. Register Complaint
2. View Complaints
3. Update Complaint Status
4. Exit
Enter your choice: 4
Exiting CivicFix System.
PS C:\Users\sdnaz\OneDrive\Desktop\ai assiant coding>
```

Justification:

I used AI to generate modular code with separate functions for each task like registering complaints, viewing data, and updating status. This makes the program easy to understand and maintain. Error handling is added to avoid crashes due to invalid input. The use of list and dictionary keeps the design simple and suitable for a prototype.

Task 4 – AI-Assisted Testing, Debugging & Optimization

PROMPT:

Review the CivicFix Python code. Generate test cases, identify bugs, and provide an optimized version with better input validation, error handling using try-except, and improved code structure.

Test Cases & Bug Identification:

1. **Test Case 1 (Valid Input):** Register complaint with valid strings. *Result: Pass.*
2. **Test Case 2 (Invalid Ticket ID):** Try to update status of Ticket ID 99 or passing a string instead of an int. *Result: Bug identified - code fails silently or crashes if a string is passed instead of an integer ID.*
3. **Test Case 3 (Empty Input):** Pass empty strings to register_complaint. *Result: Pass (handled by existing basic validation).*

Optimized Code:

```
complaints_db = []
ticket_counter = 1

def register_complaint(name: str, category: str, description: str) -> str:
    global ticket_counter
    try:
        # Optimization: Stripping whitespace
        name, category, description = name.strip(), category.strip(), description.strip()
        if not name or not category or not description:
            raise ValueError("Fields cannot be empty or just spaces.")

        complaint = {
            "id": ticket_counter,
            "name": name,
            "category": category,
            "description": description,
            "status": "Open"
        }
        complaints_db.append(complaint)
        ticket_counter += 1
        return f"Success: Ticket ID {complaint['id']} created."
    except Exception as e:
        return f"Error registering complaint: {e}"

def update_status(ticket_id: int, new_status: str) -> str:
    valid_statuses = ["Open", "In-Progress", "Resolved"]
    try:
        ticket_id = int(ticket_id) # Type enforcement optimization
        if new_status not in valid_statuses:
            raise ValueError("Invalid status provided.")

        # Optimization: using next() with a generator expression for faster lookup O(N)
        complaint = next((c for c in complaints_db if c["id"] == ticket_id), None)
        if complaint:
            complaint["status"] = new_status
            return f"Success: Ticket {ticket_id} updated."
        else:
            return "Error: Ticket ID not found."
    except ValueError as ve:
```

```
        return f"Input Error: {ve}"
    except Exception as e:
        return f"System Error: {e}"
```

Output:

(Testing Optimized Code)
Output for update_status("abc", "Resolved"): Input Error: invalid literal for int() with base 10: 'abc'

Justification:

The initial code was fragile regarding data types. If a user entered "one" instead of 1 for an ID, the system would crash. I utilized AI to help implement stricter type hinting, .strip() for input sanitation, and comprehensive try-except blocks. Additionally, changing the list traversal to a generator expression (next(...)) is a great Pythonic optimization that stops searching as soon as the ID is found.

Task 5 – Documentation, Version Control & Reflection

Task Description: Prepare the project for submission by generating a README, adding docstrings/comments, simulating a Git workflow with commit messages, and writing a reflection report.

PROMPT:

Generate a concise project README.md (Setup + Usage) for the CivicFix app. Suggest 5 standard Git commit messages simulating the development history. Finally, write a short reflection on how AI helped, limitations faced, and future improvements.

README Snippet:

CivicFix: Smart City Complaint System
A lightweight CLI application for reporting and tracking civic issues.

Setup
1. Ensure Python 3.8+ is installed.
2. Clone the repository: `git clone https://github.com/...`
3. Run the application: `python app.py`

Usage
- Developers can import `register_complaint` and `update_status` from `app.py`.
- Data is stored in-memory per session.

Simulated Git Commit Messages:

1. init: setup initial project structure and README
2. feat: add register_complaint function and core data dictionary
3. feat: implement ticket status update logic
4. fix: add input sanitation and try-except error handling
5. docs: add python docstrings to core logic functions

Justification:

Proper documentation and simulated version control are what separate a lab script from professional software. Using AI to generate the README and commit history taught me standard formatting conventions. The reflection highlights a crucial lesson: AI is a powerful assistant, but the developer must remain the "architect" to ensure the output aligns with software engineering best practices (like modularity).